

# Sferična aberacija sferičnih leč in raytracing

Ray tracing z OpenGL compute shaderji

Seminarska naloga pri predmetu Numerične metode

Avtor: Taj Šobot

Mentor: doc. dr. Rene Markovič

June 24, 2026

# 1 Uvod

Sferična aberacija je ena izmed optičnih napak, ki nastane, ko sferična leča ne lomi vseh vpadnih žarkov v isto gorišče. Dlje so vpadni žarki od optične osi leče, večji pogrešek imajo. Rezultat je popačena slika. V inštrumentih kot so kamere in teleskopi, kjer nastopajo sistemi leč pa to prevede v zamegljene slike.

V tej nalogi smo analizirali lom žarkov skozi sferično lečo, kjer je ta pojav bistveno opazljiv. Program z uporabo metode "ray tracing", simulira prehod svetlobnih žarkov skozi tako lečo. Lečo torej modelirano kot krogelna ploskev z določenim lomnim količnikom  $n$ .

## 2 Fizikalno ozadje

### 2.1 Snellov zakon

Ko svetlobni žarek prehaja iz medija z lomnim količnikom  $n_1$  v medij z lomnim količnikom  $n_2$ , velja Snellov zakon:

$$n_1 \sin \theta_1 = n_2 \sin \theta_2, \quad (1)$$

kjer je  $\theta_1$  vpadni kot in  $\theta_2$  lomni kot, oba merjena glede na normalo ploskve v točki vpada.

### 2.2 Vektorska oblika Snellovega zakona

V ray tracingu je priročno Snellov zakon izraziti vektorsko, saj poznamo smer vpadnega žarka  $\hat{d}$  in normalo ploskve  $\hat{n}$ , ne pa kotov direktno.

#### 2.2.1 Izpeljava

Označimo:

- $\hat{d}$  — enotski vektor vpadnega žarka (kaže *v* ploskev),
- $\hat{n}$  — enotski normala ploskve (kaže *stran* od ploskve, proti vpadnemu žarku),
- $\hat{t}$  — enotski vektor lomljene smeri.

Kosinuse kotov dobimo iz skalarnih produktov:

$$\cos \theta_1 = -\hat{d} \cdot \hat{n}, \quad \cos \theta_2 = \sqrt{1 - \sin^2 \theta_2}. \quad (2)$$

Iz Snellovega zakona (1) sledi:

$$\sin^2 \theta_2 = \left(\frac{n_1}{n_2}\right)^2 \sin^2 \theta_1 = \left(\frac{n_1}{n_2}\right)^2 (1 - \cos^2 \theta_1). \quad (3)$$

Lomljeni žarek  $\hat{t}$  razstavimo na tangencialno in normalno komponento glede na ploskev:

$$\hat{t} = \hat{t}_{\parallel} + \hat{t}_{\perp}. \quad (4)$$

Tangencialna komponenta je vzporedna z vpadnim žarkom (brez normalne sestavine):

$$\hat{t}_{\parallel} = \frac{n_1}{n_2} (\hat{d} + \cos \theta_1 \hat{n}). \quad (5)$$

Normalna komponenta je vzdolž normale:

$$\hat{t}_{\perp} = -\cos \theta_2 \hat{n}. \quad (6)$$

Skupaj torej:

$$\boxed{\hat{t} = \frac{n_1}{n_2} \hat{d} + \left( \frac{n_1}{n_2} \cos \theta_1 - \cos \theta_2 \right) \hat{n}} \quad (7)$$

kjer je  $\cos \theta_1 = -\hat{d} \cdot \hat{n}$  in  $\cos \theta_2 = \sqrt{1 - (n_1/n_2)^2 (1 - \cos^2 \theta_1)}$ .

### 2.2.2 Popolni odboj

Kadar je  $\sin^2 \theta_2 > 1$ , lomljenega žarka ni — nastopi popolni notranji odboj (ang. *total internal reflection*, TIR). Pogoji:

$$\left( \frac{n_1}{n_2} \right)^2 (1 - \cos^2 \theta_1) > 1. \quad (8)$$

V tem primeru žarka ne propagiramo naprej.

## 3 Geometrija sfere

### 3.1 Enačba sfere

Sfera s središčem  $\vec{c}$  in polmerom  $R$  je množica vseh točk  $\vec{P}$ , ki zadostijo:

$$|\vec{P} - \vec{c}|^2 = R^2. \quad (9)$$

### 3.2 Presečišče žarka s sfero

Žarek opišemo parametrično:

$$\vec{P}(t) = \vec{o} + t \hat{d}, \quad (10)$$

kjer je  $\vec{o}$  izhodišče žarka in  $\hat{d}$  enotska smer. Vstavimo v enačbo sfere:

$$|\vec{o} + t \hat{d} - \vec{c}|^2 = R^2. \quad (11)$$

Označimo  $\vec{oc} = \vec{o} - \vec{c}$  in razvijemo:

$$|\hat{d}|^2 t^2 + 2(\hat{d} \cdot \vec{oc}) t + |\vec{oc}|^2 - R^2 = 0 \quad (12)$$

$$t^2 + 2(\hat{d} \cdot \vec{oc}) t + |\vec{oc}|^2 - R^2 = 0, \quad (|\hat{d}|^2 = 1) \quad (13)$$

kar je kvadratna enačba  $at^2 + bt + c = 0$  s koeficienti:

$$a = 1, \quad b = 2 \hat{d} \cdot \vec{oc}, \quad c = |\vec{oc}|^2 - R^2. \quad (14)$$

Diskriminanta:

$$D = b^2 - 4c. \quad (15)$$

- Če  $D < 0$ : žarek ne seka sfere.
- Če  $D \geq 0$ : dve rešitvi  $t_{1,2} = \frac{-b \pm \sqrt{D}}{2}$ .

Izberemo najmanjši pozitivni  $t$  (najbližje presečišče pred žarkom):

$$t = \min\{t_1, t_2 \mid t > \varepsilon\}, \quad (16)$$

kjer je  $\varepsilon > 0$  majhna vrednost, ki prepreči samopresečišče (ang. *self-intersection*).

### 3.3 Normala v točki presečišča

Ko dobimo točko presečišča  $\vec{P}_{hit} = \vec{o} + t \hat{d}$ , je zunanja normala sfere:

$$\hat{n} = \frac{\vec{P}_{hit} - \vec{c}}{|\vec{P}_{hit} - \vec{c}|} = \frac{\vec{P}_{hit} - \vec{c}}{R}. \quad (17)$$

Pri izstopu iz sfere (druga površina) moramo normalo obrniti:  $\hat{n} \rightarrow -\hat{n}$ , da pravilno označimo prehod iz gostejšega v redkejši medij.

## 4 Sferična aberacija

### 4.1 Paraksijska aproksimacija

Za žarke blizu optične osi (majhen  $\theta_1$ ) velja  $\sin \theta \approx \theta$ , kar dá enostavno lensmaker's enačbo:

$$\frac{1}{a} + \frac{1}{b} = \frac{1}{f}, \quad (18)$$

kjer je  $a$  razdalja predmeta,  $b$  razdalja slike in  $f$  goriščna razdalja leče.

### 4.2 Izvor aberacije

Za žarke daleč od osi paraksijska aproksimacija ne velja več. Dejansko gorišče žarka, ki vpade na rob sferne leče z polmerom  $R$  in lomnim količnikom  $n$ , je bližje leči kot paraksijsko gorišče. Razlika med paraksijskim in dejanskim goriščnim mestom tvori *sferno aberacijo*.

Za sferično lečo (obe površini sta krogelni s polmerom  $R$ ) je goriščna razdalja v paraksijski aproksimaciji:

$$\frac{1}{f} = (n - 1) \left( \frac{1}{R_1} - \frac{1}{R_2} \right), \quad (19)$$

kar je *lensmaker's enačba* za tanko lečo. Za sferno kroglo z  $R_1 = R$  in  $R_2 = -R$ :

$$\frac{1}{f} = \frac{2(n - 1)}{R}. \quad (20)$$

## 5 Implementacija v GLSL

### 5.1 Compute shader — presečišče s sfero

Vsak invokacijski thread obdeluje en piksel izhodne slike. Žarek je metamo vzporedno z optično osjo ( $z$ -smeri), izhodišče žarka pa ustreza položaju piksla v ravnini senzorja.

Listing 1: Presečišče žarka s sfero v GLSL

```
float intersectSphere(vec3 origin, vec3 dir, vec3 center, float r)
{
    vec3 oc = origin - center;
    float b = 2.0 * dot(dir, oc);
    float c = dot(oc, oc) - r * r;
    float D = b * b - 4.0 * c;           // a = 1 ker je |dir| = 1
    if (D < 0.0) return -1.0;
    float t1 = (-b - sqrt(D)) / 2.0;
    float t2 = (-b + sqrt(D)) / 2.0;
    if (t1 > 0.001) return t1;
    if (t2 > 0.001) return t2;
    return -1.0;
}
```

### 5.2 Compute shader — vektorski lom

Implementacija enačbe (7):

Listing 2: Vektorski Snellov zakon v GLSL

```
vec3 refractRay(vec3 d, vec3 normal, float n1, float n2) {
    float cosI = -dot(normal, d);
    float sinT2 = (n1/n2)*(n1/n2) * (1.0 - cosI*cosI);
    if (sinT2 > 1.0) return vec3(0.0); // TIR
    float cosT = sqrt(1.0 - sinT2);
    return (n1/n2)*d + (n1/n2*cosI - cosT)*normal;
}
```

### 5.3 Potek žarka skozi lečo

Vsak žarek preide skozi dve površini sfere — vstopno in izstopno. Na vsaki površini apliciramo lom:

Listing 3: Zanka dveh prehodov skozi sfero

```
for (int i = 0; i < 2; i++) {
    float t = intersectSphere(rayOrigin, rayDir, lensCenter, u_r)
    ;
    if (t <= 0.0) break;

    vec3 hit      = rayOrigin + t * rayDir;
    vec3 normal    = normalize(hit - lensCenter);
    if (i == 1) normal = -normal; // izstop: normala obrnjena
    navznoter
}
```

```

float n1 = (i == 0) ? 1.0 : u_n;
float n2 = (i == 0) ? u_n : 1.0;

rayDir    = refractRay(rayDir, normal, n1, n2);
rayOrigin = hit + rayDir * 0.001; // epsilon odmik
}

```

Na prvi površini ( $i = 0$ ) gre žarek iz zraka ( $n_1 = 1$ ) v steklo ( $n_2 = n$ ), na drugi ( $i = 1$ ) pa iz stekla ( $n_1 = n$ ) nazaj v zrak ( $n_2 = 1$ ). Normala na izstopu je obrnjena, ker mora pri vektorski obliki Snellovega zakona normala kazati *stran od medija, v katerem je vpadni žarek*.

## 5.4 Vzorčenje slike na zaslonu

Po izstopu iz leče žarek propagiramo do ravnine slike na razdalji  $u_b + u_a$ :

Listing 4: Vzorčenje objektne ravnine

```

float t_plate = (objectPos.z - rayOrigin.z) / rayDir.z;
vec3 hit      = rayOrigin + t_plate * rayDir;

// UV koordinate za vzorčenje vhodne teksture
vec2 uv = (hit.xy - objectPos.xy) / u_ObjScale;
if (all(greaterThanEqual(uv, vec2(0.0))) &&
    all(lessThanEqual(uv, vec2(1.0))))
    color = texture(u_input, uv);
else
    color = vec4(0.2 * normalize(hit.xyy), 1.0);

```

## 6 Parametri programa

Program sprejema naslednje uniforme, ki jih med izvajanjem nastavljamo z bližnjicami:

| Uniform    | Pomen                        | Tippična vrednost                            |
|------------|------------------------------|----------------------------------------------|
| u_r        | Polmer sferne leče           | 0.5 ... 2.0                                  |
| u_n        | Lomni količnik stekla        | 1.5 (crown glass)                            |
| u_a        | Razdalja predmeta od leče    | 10.0 (vzporedni žarki $\rightarrow \infty$ ) |
| u_b        | Razdalja slike od leče       | nastavljivo                                  |
| u_xLensPos | Horizontalni položaj leče    | 0.0                                          |
| u_yLensPos | Vertikalni položaj leče      | 0.0                                          |
| u_xObjPos  | Horizontalni položaj objekta | 0.0                                          |
| u_yObjPos  | Vertikalni položaj objekta   | 0.0                                          |
| u_ObjScale | Velikost objekta na zaslobu  | 1.0                                          |

Table 1: Uniformni parametri compute shaderja

## 7 Zaključek

Implementirali smo ray tracer za simulacijo sferične aberacije, ki deluje v realnem času na GPU. Vsak piksel izhodne slike ustreza enemu svetlobnemu žarku, ki ga propagiramo skozi sferno lečo z dvema lomoma (vstop in izstop), pri čemer na vsaki površini apliciramo vektorsko obliko Snellovega zakona. Sferična aberacija se jasno pokaže kot razmazanost slike, ki se povečuje z večjim polmerom žarka in večjim lomnim količnikom. Interaktivna nastavitve parametrov omogoča intuitivno razumevanje, kako posamezni parametri vplivajo na kakovost slike.

## 8 Viri

### References

- [1] E. Hecht, *Optics*, 5th ed., Pearson, 2017.
- [2] P. Shirley, *Ray Tracing in One Weekend*, 2020.  
Dostopno na: <https://raytracing.github.io/>
- [3] Khronos Group, *GLSL Specification 4.60*.  
Dostopno na:  
<https://registry.khronos.org/OpenGL/specs/gl/GLSLangSpec.4.60.pdf>
- [4] Khronos Group, *OpenGL 4.3 Core Profile Specification*, 2012.  
Dostopno na:  
<https://registry.khronos.org/OpenGL/specs/gl/glspec43.core.pdf>